

UDAAN 2026 — Issue Audit

Audited: 30 July 2026 · Next.js 14.2.5 · App Router · 23 source files / ~2,325 lines **Scope:**
C:\Users\navne\Desktop\UDAAN\Udaan copy **Baseline:** npx tsc --noEmit passes clean. npx next lint emits 5 warnings (all no-img-element).

48 issues found. Every item below was verified against the source — file and line references are exact, and byte sizes / image dimensions were read from the files themselves.

P0 — Root causes of "it lags and stops working"

These four are, together, the entire performance complaint. Nothing else on this list matters until they are fixed.

1. A setInterval re-renders the header ~2.2× per second, forever

src/components/navigation.tsx:23-29

```
const interval = setInterval(() => {
  setFrameIndex((prev) => (prev + 1) % udaanFrames.length)
}, 450)
```

This drives the decorative Udaan' 26 → Udaaaaaaaaaan' 26 stretching text. Because <Navigation/> lives in the root layout (src/app/layout.tsx:31), this loop runs on **every route, in every open tab, for the entire session** — it never stops, and it keeps burning CPU in background tabs. **This is the single biggest cause of the lag.**

The effect itself is purely visual and can be reproduced with a CSS keyframe animation at zero JS and zero re-renders.

2. ~2.1 MB of unoptimised images on the homepage

All images are raw tags. There is no next/image anywhere, so nothing is resized, converted, or lazy-loaded. sharp is not installed either, so next start could not optimise them even if it wanted to.

File	On disk	Actual pixels	Displayed at	Problem
sgsits_logo.png	839 KB	1024×766 RGBA	64–96 px tall	~46× more pixels than needed
udaan_stage.png	676 KB	633×471	~1152×500	Upscaled 1.8× — blurry and huge
udaan_group.png	589 KB	597×399	~370×208	2.6× oversized

Homepage total ~2.1 MB

All four are colour-type 6 (RGBA) PNGs — the worst possible format for photographs. udaan_stage.png alone should be ~55 KB as WebP.

Note the hero is *both* problems at once: it is the largest single asset **and** it looks soft, because a 633 px-wide source is being stretched across a 1152 px container ([src/components/hero.tsx:12-18](#)).

3. Sections render at `opacity: 0` until JavaScript proves otherwise

Seven sections gate their own visibility on a hydrated IntersectionObserver:

```
initial={{ opacity: 0, y: 15 }}
animate={inView ? { opacity: 1, y: 0 } : { opacity: 0, y: 15 }}
```

[about.tsx:40](#) · [document-viewer.tsx:114](#) · [medalist-directory.tsx:188](#) · [schedule.tsx:62](#) · [gallery.tsx:40](#) · [faq.tsx:54](#) · [registration.tsx:20](#)

If hydration is slow, fails, or the observer never fires, **the content stays invisible** — a blank page with a working scrollbar. This is the most likely explanation for "stops working". Content should be visible by default and animation should be the enhancement, never the gate.

4. Every section is a client component, for animation it doesn't need

[framer-motion](#) (~50 KB gz) is imported in **10 files**; [react-intersection-observer](#) in **7** — all to produce simple fade-ins. Even [src/app/page.tsx:1](#) and [src/app/udaan/page.tsx:1](#) are marked "use client" despite holding no state whatsoever.

[about.tsx](#), [schedule.tsx](#), [registration.tsx](#) and [faq.tsx](#) have no genuine interactivity requirement ([faq.tsx](#) needs only a native `<details>`), so the entire homepage currently ships and hydrates JS it has no use for.

P1 — Broken functionality

5. [dresscode.docx](#) is not a Word document and will not open

[public/docs/dresscode.docx](#) — 381 bytes. First bytes on disk:

```
UDAAN 2025 - SGSITS INDORE DRESS CODE GUIDELINES\r\n\r\nMALE CANDIDATES:\r\n-
White or Off-White Kurta Pyj...
```

That is **UTF-8 plain text with a renamed extension**. A real `.docx` is a ZIP archive and must begin with `PK\x03\x04`. Word will reject this file. It is linked from two places — [document-viewer.tsx:87](#) and [registration.tsx:111](#).

6. All three PDFs have fabricated cross-reference tables

[scripts/generate-docs.js:31-41](#) writes hardcoded placeholder byte offsets:

```
stream.push('0000000009 00000 n ');
stream.push('0000000056 00000 n '); // ← invented, not measured
```

```
...  
stream.push('startxref');  
stream.push('400'); // ← invented
```

The real offsets are never computed. Chrome's lenient parser silently repairs these; Acrobat and other strict viewers can refuse them outright. Affects `brochure.pdf`, `schedule.pdf` and `circular.pdf`.

7. The two registration API routes use separate, non-persistent stores

`src/app/api/registrations/route.ts:4` and `src/app/api/registrations/[id]/route.ts:4` **each declare their own** module-level array:

```
let registrations: any[] = []
```

They are different variables in different modules. `PATCH /api/registrations/[id]` can therefore never find a record, and either array resets on every cold start. Meanwhile `POST /api/register (route.ts:25)` validates its input and then only `console.logs` it — nothing is stored. `/api/stats` returns hardcoded invented numbers.

Nothing in the UI calls any of these except the admin dashboard reading `/api/stats`; real registration goes to a Google Form (`registration.tsx:15`).

8. Countdown flashes `00:00:00:00` on every page load

`src/components/countdown-timer.tsx:6-11` initialises to all zeroes and the first `setInterval` tick is a full **1000 ms** later, so every visitor sees a zeroed clock before it snaps to the real value. State should be computed on initialisation.

The same component has no terminal state: at `distance <= 0` (line 21-24) it just clears the interval, leaving the last stale numbers frozen on screen rather than showing that the ceremony has arrived.

9. `/udaan` is a byte-identical duplicate of the homepage

`src/app/udaan/page.tsx` renders exactly the same eight sections as `src/app/page.tsx`. The nav's "UDAAN" link (`navigation.tsx:33`) therefore leads to a clone of the page you are already on. Costs double the build output, splits the `#documents` / `#medalists` anchors across two URLs, and reads as duplicate content to search engines. The two also disagree cosmetically — `/udaan` has `pt-12`, `/` does not.

10. Dark mode leaves a cream page background

`src/app/layout.tsx:24`

```
<body className="font-sans antialiased bg-[#FAF8F5] text-[#1C2430]">
```

Hardcoded light colours with **no dark: variant**. Sections paint over it, so the bug hides until you overscroll or hit any gap between sections — then the light background shows through underneath a dark page.

11. `enableSystem` is set but can never take effect

`src/app/layout.tsx:26-29` passes `defaultTheme="light"` alongside `enableSystem`. With an explicit non-`"system"` default, a first-time visitor's OS dark-mode preference is ignored. Separately, `navigation.tsx:94,98,109,113` branches on `theme` rather than `resolvedTheme`, so the sun/moon icon can disagree with what is actually rendered.

12. Missing `download` attribute on the schedule link

`src/components/schedule.tsx:122` — every other document link in the project sets `download` (`document-viewer.tsx:177`, `registration.tsx:104,112`, `about/page.tsx:37`). This one opens in-tab instead.

13. `next.config.js` image config is entirely dead

```
images: { domains: [ 'udaan.sgsits.ac.in' ] }
```

`domains` is deprecated in Next 14 (superseded by `remotePatterns`); the listed host is never used as an image source; and since no `next/image` exists in the codebase, the block has no effect at all. The four **remote Unsplash images** that *are* loaded (`gallery.tsx:19-22`) are not configured — they are plain `<img>` tags, so the gallery stalls on a third-party host and breaks entirely offline.

14. No favicon

There is no `app/icon`, no `favicon.ico`, and no `icons` entry in `metadata` (`src/app/layout.tsx:11-15`). Every single page load 404s on `/favicon.ico`. Also missing: `metadataBase` and any OpenGraph tags, so shared links have no preview.

P2 — Accessibility and UX

15. Gallery lightbox is a keyboard and scroll trap

`src/components/gallery.tsx:78-109`. The overlay has:

- no Esc to close
- no `←/→` arrow navigation (despite having prev/next buttons)
- no click-outside-to-dismiss
- **no body scroll lock** — the page scrolls freely behind the open lightbox
- no focus trap and no focus restore on close
- no `role="dialog"` / `aria-modal`
- three icon-only buttons with no accessible name (lines 80, 86, 92)

16. FAQ accordion is invisible to assistive tech

`src/components/faq.tsx:70-89` is `<div>` + `<button>` with conditional rendering — no `aria-expanded`, no `aria-controls`, no region association. A native `<details>/<summary>` would be correct, accessible for free, **and** ship zero JS.

17. Mobile menu has no accessible state and stays open

`src/components/navigation.tsx:116-140` — no `aria-expanded` on the toggle, no Esc handler, no scroll lock, and it does not close on route change (only on direct link click, line 134).

18. Header is not sticky

The ~220 px header scrolls away and never comes back, so navigating from the bottom of a long single-page site means scrolling all the way back to the top. There is also no `scroll-margin-top` on any section, which becomes a live bug the moment the header is made sticky.

19. Header logo is absolutely positioned over centred text

`src/components/navigation.tsx:46` — `absolute left-4 md:left-8 top-2` layered over a `text-center` title. It survives at common widths but is fragile around 768–900 px, where a 96 px seal and a `tracking-[0.25em]` heading compete for the same row.

20. Every image causes layout shift

No `width/height` (or `next/image`) on any of the five `<img>` tags, so each one reflows the page as it decodes. The hero — the LCP element — also has no `priority` hint, so the browser discovers the largest asset on the page late.

21. `container` fights the explicit gutters

`tailwind.config.ts:11-17` sets `container: { padding: "2rem" }`, but every usage writes `container mx-auto px-4` (e.g. `hero.tsx:56`, `about.tsx:39`, `faq.tsx:53`). The two padding declarations have equal specificity, so the winner depends on stylesheet order — producing inconsistent gutters between sections.

22. No `prefers-reduced-motion` support

Nothing in `globals.css` or any component respects it, and `html { scroll-behavior: smooth }` (`globals.css:75-77`) is applied unconditionally.

23. Filter re-runs and re-animates on every keystroke

`src/components/medalist-directory.tsx:173` recomputes the filter on every render with no `useMemo` and no debounce. Worse, line 258 applies framer-motion `layout` to **every card**, so each keystroke triggers a full measure-and-animate pass across the grid.

P3 — Dead weight and code health

24. `sgsits_seal.png` is a byte-identical duplicate

839 KB, md5 `bd51f43393e95507404017a367a540a1` — **identical** to `sgsits_logo.png`, and referenced nowhere in `src/`.

25. Two more unreferenced assets

`ref_layout.jpg` (250 KB — it is the design mock-up of the reference site) and `qr.svg` are both unused. With #24, that is ~1.1 MB of dead files in `public/images/`.

26. Four unused npm dependencies

`react-hook-form`, `@hookform/resolvers`, `date-fns` — **zero** imports anywhere in `src/`. Leftover scaffolding from a registration form that was replaced by a Google Form link. `clsx` + `tailwind-merge` are used only by `src/lib/utils.ts`, whose `cn()` helper is itself never called — so `src/lib/utils.ts` is dead too.

27. Seven unused imports in one line

`src/app/about/page.tsx:6` imports `Award`, `Trophy`, `Users`, `ShieldCheck`, `Download`, `ArrowRight`, `Building2` — none are used. Only `MapPin` and `Shirt` are.

28. Unused CSS and Tailwind config

`globals.css:67-72` defines a `.glass-panel` glassmorphism utility that nothing uses (and which would clash with the editorial design anyway). `tailwind.config.ts:63-79` defines `accordion-down` / `accordion-up` keyframes and animations for Radix UI — a library the project does not depend on.

29. The admin page looks like a different website

`src/app/admin/page.tsx` is the **only** file that uses the design tokens (`bg-card`, `text-muted-foreground`, `border`) plus `rounded-xl` corners and a purple accent (`bg-purple-500`, line 134). Every other page hardcodes hex and uses square corners. Its stats are also invented, and its "Recent Registrations" table (lines 170-202) is three rows of hardcoded HTML dated 2025.

30. 467 hardcoded hex colours across 20 shades

The real maintainability problem, and the reason dark mode is inconsistent. `globals.css:5-49` already defines a complete HSL token set, and `tailwind.config.ts:18-60` already maps it to `bg-background` / `text-accent` / `border-border` — **and almost nothing uses it**. Instead:

Count	Hex	Meaning
79	#142338	navy — headings, buttons
66	#C5A059	gold — accent
61	#212B3B	dark-mode border
52	#EAE5DC	light border
39	#FAF8F5	cream background
35	#64748B	muted text
31	#1C2430	dark-mode card
22	#CBD5E1	dark-mode body text
21	#475569	body text
20	#12161E	dark-mode background

Count	Hex	Meaning
15	#94A3B8	dark-mode muted
9	#E4DFD7	section divider
6	#0F1B2B	button hover
3+	#334155 #DCD5C9 #A89882 #B0A798 #7A6B58 #2A3649 #1B2A4A	long tail

Nearly every colour is written twice — once light, once as a **dark:** variant — so a single token swap replaces pairs like `text-[#142338] dark:text-[#FAF8F5]` → `text-foreground`.

31. **any** types in the API layer and admin page

`admin/page.tsx:8` (`React.useState<any>`), `api/registrations/route.ts:4`, `api/registrations/[id]/route.ts:4`. `strict: true` is on in `tsconfig.json`, so these are deliberate escape hatches rather than oversights — but they hide the shape of the data.

Two content inconsistencies — flagged, not touched

You asked that content not change, so these are listed for your decision only:

1. **The documents are last year's.** All four files in `public/docs/` say "UDAAN 2025 ... October 8, 2025 ... 10:00 AM - 1:00 PM", while the site says "Thursday, 27 August 2026" and "09:00 AM – 04:00 PM IST". The text lives in `scripts/generate-docs.js`.
2. **Medalist data is placeholder.** `medalist-directory.tsx:22-150` contains 10 invented students (Aditya Sharma, Ananya Verma, ...) with fabricated enrollment numbers and CGPAs, and `api/stats/route.ts:4-22` returns invented registration counts. These must be replaced with real data before the site goes live.

Summary

Priority	Count	Theme
P0	4	The lag and the blank-page failures
P1	10	Broken files, dead APIs, dark mode, favicon
P2	9	Keyboard access, sticky nav, layout shift
P3	8	~1.1 MB dead assets, 4 unused deps, 467 hex literals
<i>Flagged</i>	2	Content decisions left to you

The good news: `tsc --noEmit` is already clean, lint produces only the 5 image warnings, and the codebase is small (23 files). The four P0 items are all self-contained and account for essentially the whole "lags and stops working" complaint.